

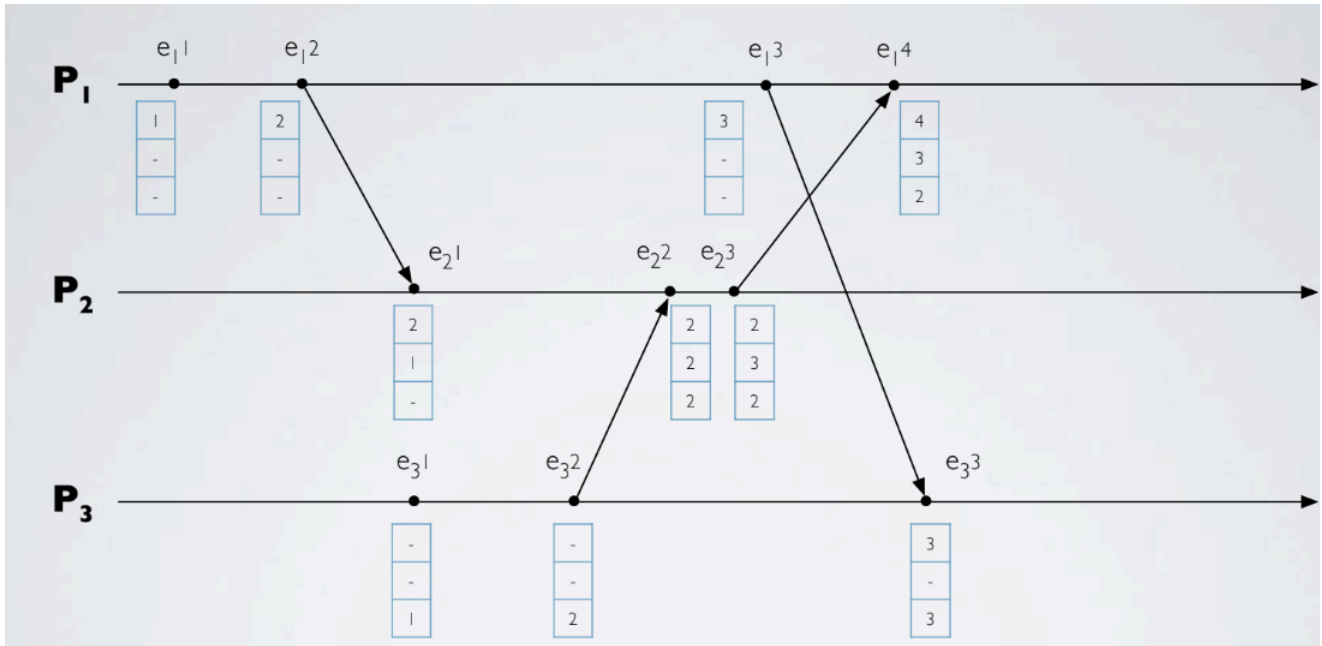
Distributed Systems - 6_10

Let's take a look again at Vector clock.

We say that when two events (e, e') are concurrent that $e \parallel e'$.

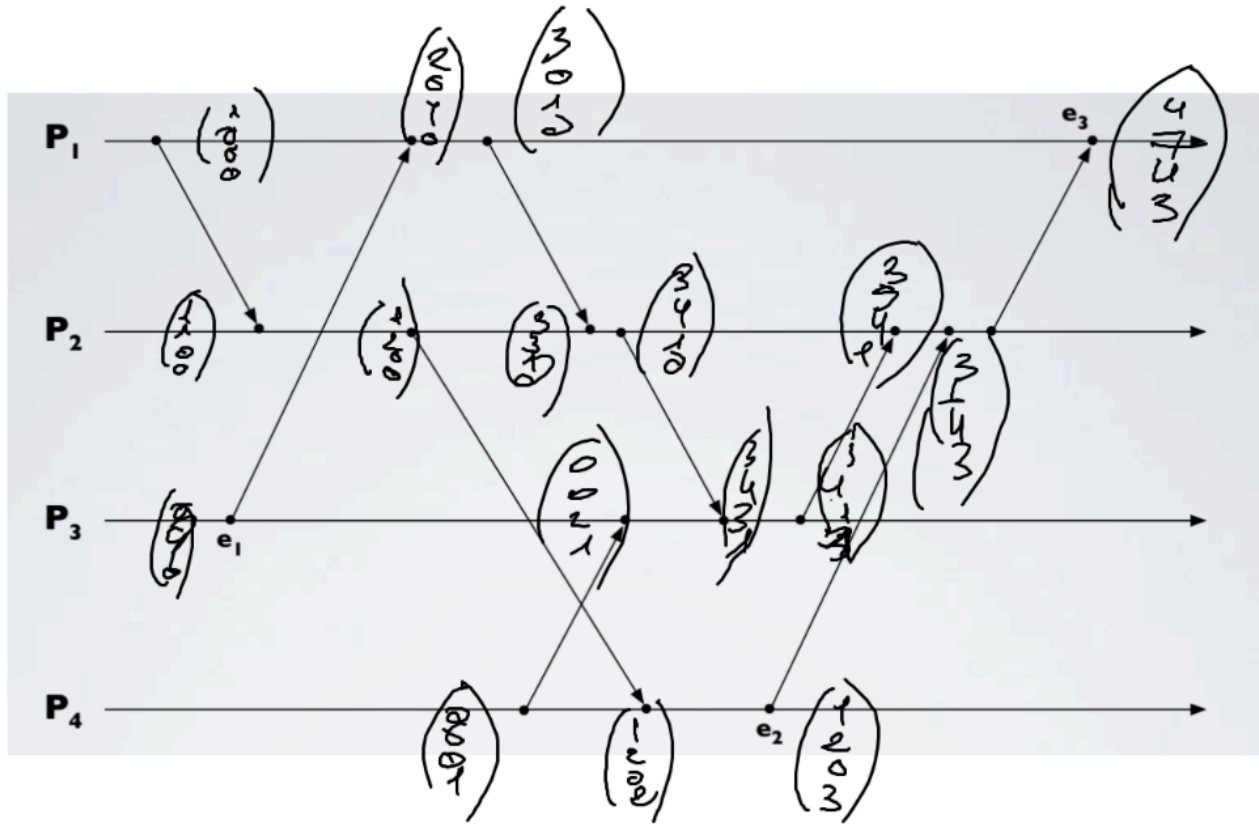
Let's take a look at an example:

Remember that, when updating, we have to take the maximum of each value only if $e \rightarrow e'$.



We see that, for example, $e_{1.3}$ and $e_{2.2}$ are concurrent.

Let's try an exercise:



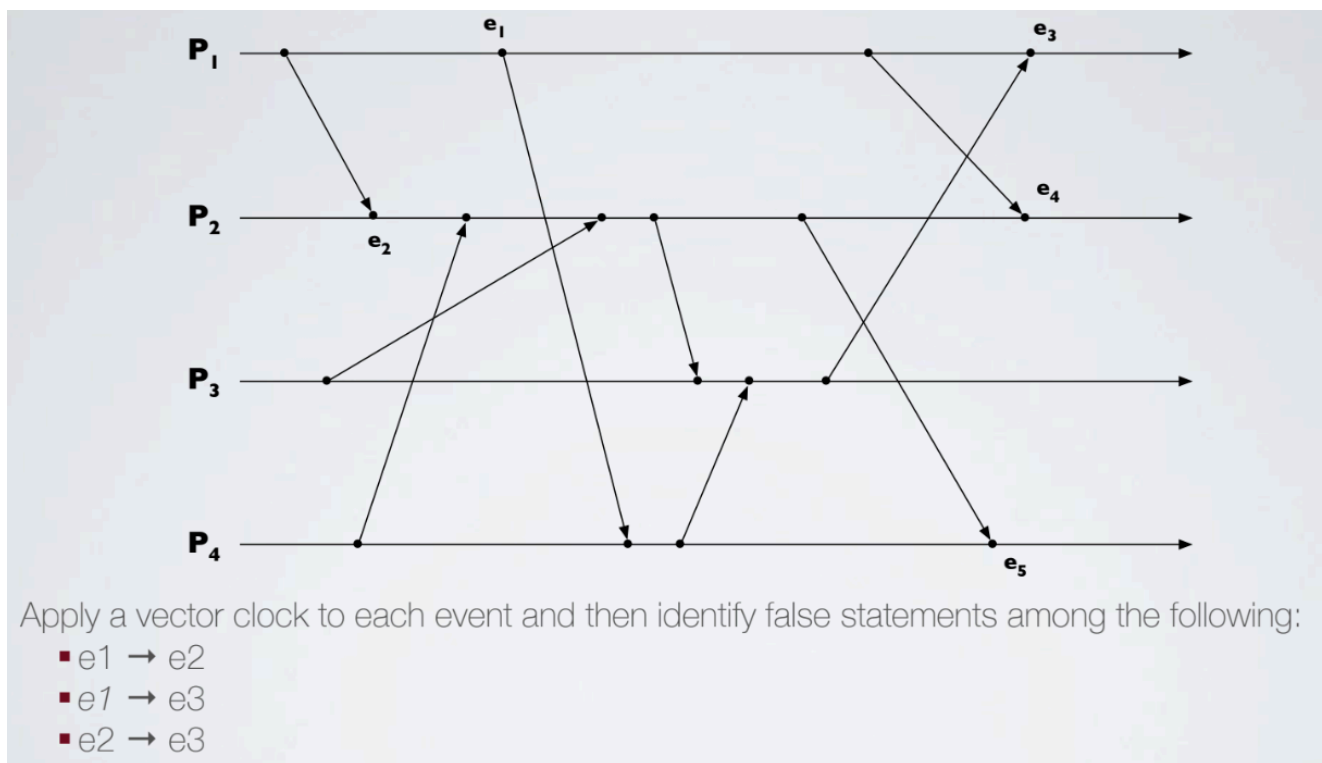
We also know that " $e_1 \rightarrow e_2$ " is false because we cannot compare the clocks, thus they are concurrent.

" $e_1 \rightarrow e_3$ " instead is true, there is a path of "happened before" relationship and the vectors are comparable.

" $e_2 \rightarrow e_3$ " is also true, vectors are comparable.

② Exercise

Do it by yourself.



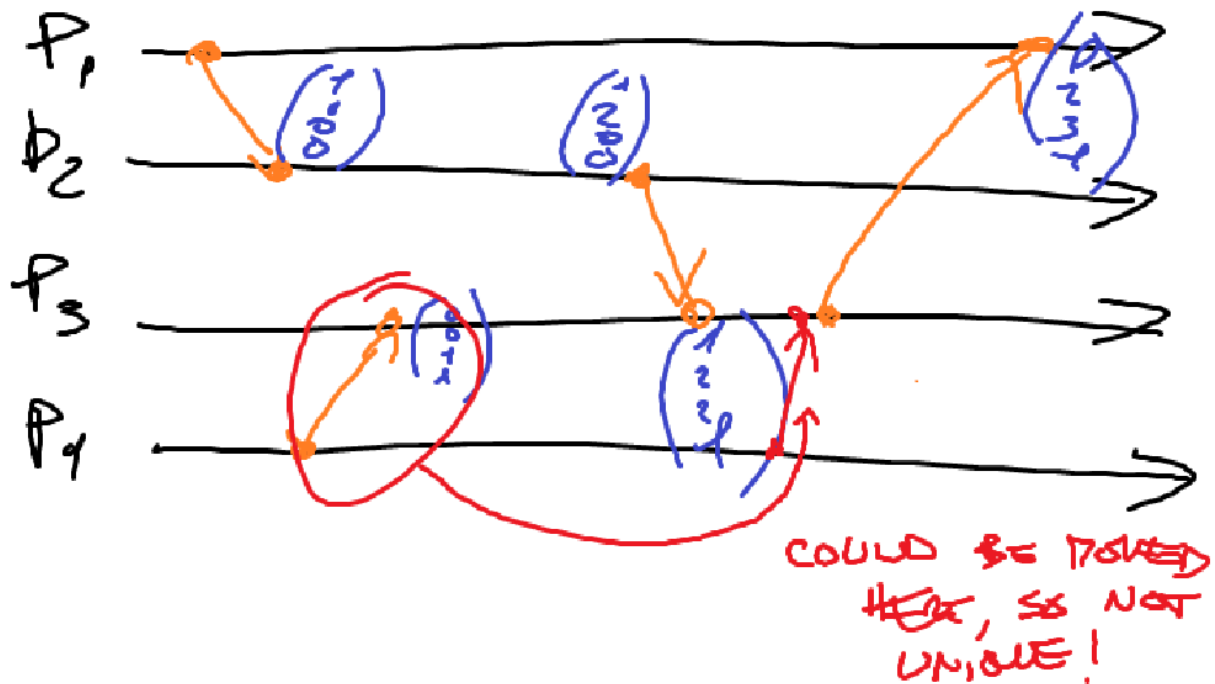
We've seen two methods of measuring logical time. We can use them to solve different problems. As for the vector clock, we'll see how to solve the causal broadcast.

As for the scalar timestamp, we'll see the Lamport's mutual exclusion.

But before, let's check an exercise.

- Consider an asynchronous message passing system that uses vectors clock to implement some causal consistency check. The message passing system is composed by 4 processes with IDs 1 to 4, and, as usual, the ID is used as displacement in the vector clock (i.e., the locations are (p_1, p_2, p_3, p_4)). Vector clocks are updated increasing before send. Processes communicate by point2point links. You start debugging process p_1 (the process with id 1) in the middle of the algorithm execution. You see the following stream of messages exiting and entering the ethernet card of process p_1 :
- Time 00:00 – EXITING: Send Message [MSG CONTENT] Vector Clock: (1, 0, 0, 0)
- Time 00:05 – ENTERING: Rcvd Message [MSG CONTENT] Vector Clock: (1, 2, 3, 1)
- Q3.1: Draw an execution that justifies the vectors clocks you are seeing. Is such an execution unique?
- Q3.2: There exists an execution that justifies the vector clocks and where there exists at least a process that does not send any message? Justify your answer.

The exercise says that the processes use vector clock.



Regarding question 2, we cannot do it because each component is at least 1, so we know that each process has at least a "sending" event.

Now we see the Lamport's mutual exclusion. It does not tolerate crash, so it works only when all processes are correct.

We'll use a FIFO link. It has the properties of a perfect link (PL1, PL2, FL3), as well as a new property FIFO: if a process p delivers m before m' , then m was sent before m' .

The algorithm works with a pending set, in which we put messages. We also have a sequence number, and a delivery sequence number.

We instantiate an object for each process we want to communicate with.

Algorithm 5 FIFO Links - client p_i

```
1: upon event INIT
2:    $Pending = \emptyset$ 
3:    $seq = 0$ 
4:    $del\_seq = 0$ 

5:  $\triangleright$ 
6: upon event SEND FL( $\langle p_d, m \rangle$ )
7:   SEND PERFECT( $p_d, \langle m, seq \rangle$ )
8:    $seq = seq + 1$ 

9: upon event DELIVERY FROM PERFECT( $\langle p_s, \langle m, seq \rangle \rangle$ )
10:   $Pending = Pending \cup \{\langle p_s, \langle m, seq \rangle \rangle\}$ 

11: upon event  $\exists \langle p_s, \langle m, send\_seq \rangle \rangle \in Pending$  SUCH THAT  $send\_seq = del\_seq$ 
12:  DELIVERY FL( $p_s, m$ )
13:   $Pending = Pending \setminus \{\langle p_s, \langle m, send\_seq \rangle \rangle\}$ 
14:   $del\_seq = del\_seq + 1$  typo: here you have to increment del\_seq
```

Why does the algorithm work? The algorithm cannot invert two messages. let's reason by contradiction. Let's say that at time $t = 1$ we deliver $(m, 5)$ and at $t = 3$ we deliver $(m, 3)$. It could not work because we would have that del_seq would have to be 5 at $t = 1$. But when do we update the value? In the last handler, and this means that at time $t - 1$ we would have to have 4, and to have 4 we would have to have 3 set before, meaning, that we've already received msg 3.

Now let's actually take a look at the mutual exclusion algorithm.

We have another event known as Request, as well as Grant and Release.

small missing part

What are the properties?

- Mutual exclusion: at time t , only one process is inside the critical section, which means that only one process has received a grant for the critical section, and no one else has received it.
- Liveness: if a process p requests access, then it eventually enters the CS.
- Fairness: If the request of process p happens before the request of process q , then q cannot access the CS before p . *If I make the request before you in logical time, then I will enter before you.*

To build this, we'll build it upon a perfect point2point link, plus a FIFO property.

The algorithm assumes no crashes ($F = 0$). When a process wants to enter the CS it sends a request message to all the other.

Let's see the algorithm.*

```

1: upon event INIT
2:    $Requests = Acks = \emptyset$ 
3:    $scalar\_clock = 0$ 
4:    $my\_req = \perp$ 
5:    $\Pi = \{p_0, p_1, \dots, p_{n-1}\}$  ▷ Set of all processes

6: ▷ Request access to CS from upper layer
7: upon event REQUEST
8:    $scalar\_clock = scalar\_clock + 1$ 
9:    $my\_req = (REQ, ts = \langle i, scalar\_clock \rangle)$ 
10:  for all  $p_j \in \Pi$  do
11:    SEND FIFOPERFECTLINK( $p_j, req\_msg$ )  ▷ Send a REQ containing my ID (i) and ts (scalar_clock) to all
     $p \in \Pi$ 

12: ▷ Release CS from upper layer
13: upon event RELEASE
14:    $my\_req = \perp$ 
15:    $scalar\_clock = scalar\_clock + 1$ 
16:   for all  $p_j \in \Pi$  do
17:     SEND FIFOPERFECTLINK( $p_j, (RLS, ts = \langle i, scalar\_clock \rangle)$ )

18: ▷  $ts(x) < ts(y)$  when  $scalar\_clock$  of  $x$  is less than the one of  $y$ , or they are equal and the id that sent  $x$  is less
    than the id that sent  $y$ 
19: upon event  $\nexists req \in Requests : ts(req) < ts(my\_req) \wedge \forall p \in \Pi : \exists m \in Acks | ts(m) > ts(my\_req) \wedge sender(m) = p$ 
20:   trigger event GRANTED

21: upon event DELIVER MESSAGE( $m$ )
22:    $scalar\_clock = \max(clock(m), scalar\_clock) + 1$ 
23:   if  $m$  is a REQ then
24:      $Request\_set = Request\_set \cup \{m\}$ 
25:      $scalar\_clock = scalar\_clock + 1$ 
26:     SEND FIFOPERFECTLINK( $sender(m), (ACK, ts = \langle i, scalar\_clock \rangle)$ )
27:   else if  $m$  is a ACK then
28:      $Acks = Acks \cup \{m\}$ 
29:   else if  $m$  is a RLS  $\wedge \exists req \in Request\_set : sender(req) = sender(m)$  then
30:      $Requests = Requests \setminus \{req\}$ 

```

**

We have an init event, in which each process creates a request set and an Ack set. The requests set contains all request from others processes, and then we ack the requests.

We then have a variable of scalar clock, used as Lamport's clock.

The value of my_req is being set to null ($\perp$).

Then I also define a set of all the processes in a network Π .

We then have a request handler, we trigger when we do a request to enter the critical section.

Then we have a release, the handler i trigger when I release the critical section.

Then another that takes a look at requests, and grants us access to the critical section.

To conclude, an handler that processes received messages.

As for messages, we have three possibilities: request, release and ack.

Let's say that I want to do a request. I will execute the second handler, by

increasing the scalar clock, then I create a my request message, which is given by (*Req*, followed by a tuple of (id, scalar clock), so like (*REQ*, (1, 1)). If two requests have the same scalar clock, then the request that wins is given by the one with the smallest id. Also if two requests have the same id, then the one with the smallest scalar clock wins.

The first component of message is given by REQ so that the one who receives it is able to understand that it's a request.

Then we create a broadcast message by sending it to all processes.

Let's jump to the delivery handler. We update the scalar clock of the process who has received the message, by taking the maximum value + 1.

Then we check what kind of message it is. If the message is a request, we update the request set, we increase the scalar clock and then send an ACK.

When the other process receives the message, it activate the same handler and does some actions, updating it's ack set.

When can process p_1 enter the CS? With the third handler, such that there doesn't exist a request in the set such that the timestamp of the request is smaller than the one actually saved and, for each process, there exists a message m in the ack set (we received an ack for everyone), and the timestamp for each element is greater than the one for myreq.

Let's say that p_1 can enter the critical section. What happens is that GRANTED is granted. At the end he will release the critical section, triggering another handler.

We set my_req = $\perp$, we increase the logical clock and send a message to everyone by saying that the CS has been released, indicating the timestamp as well.

When a process receives the message he enters the handler for deliver. When I receive a release I check if there's a request in the request set...

Let's see an example.

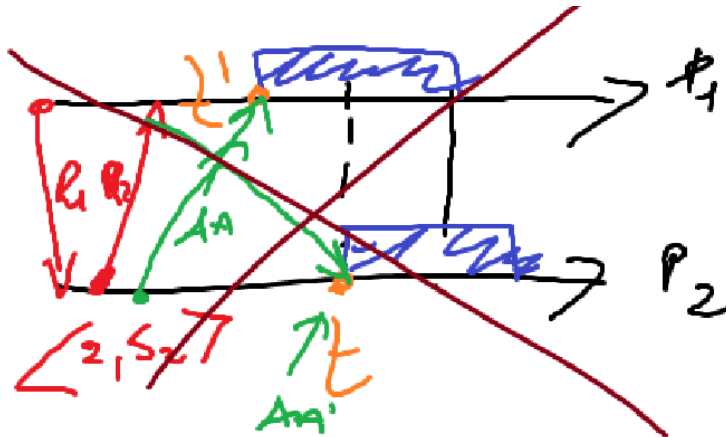
p_3 wants to enter the CS in other processes, thus he sends a msg to everyone (*REQ*, (3, 3)). When p_2 receives the message, thus increasing the scalar clock to 4, then increase it to 5 by sending an ACK back to p_3 . Now also p_1 wants to access the CS. He creates a message, with id 1 and scalar clock equal to 6. When others receive the message, they send the ACK, not checking if someone is in the critical section. The ack will contain 8 as timestamp (because I received 6, then +1=7). Then all processes has all the ACKs, and p_3 can enter the CS. At some point he releases it, thus creating a release message.

This algorithm works on the total order of timestamps.

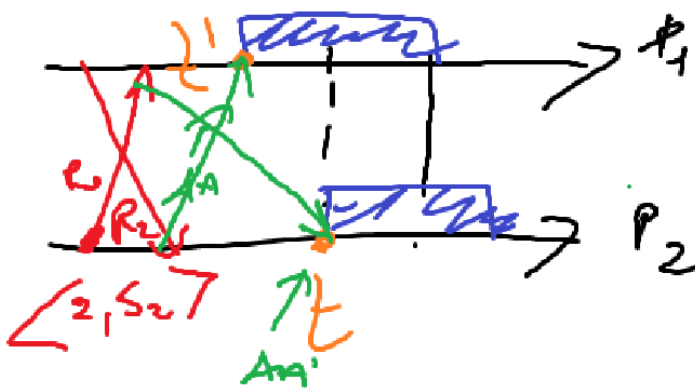
When we want to access the CS, we'll have a total number of messages given by

$3n$ (n processes, thus n messages, and n acks, plus n release messages).

Now let's see if this holds. We assume by contradiction that two processes enter the CS at the same time. We have p_1 and p_2 entering at the same time in the CS. Let's say that p_2 enters at a time t , then he must have received the ACKs from all processes and after it sent the request message.



AA = all ACKs



AA = all ACKs

Now let's show the fairness property and it's proof.

Let us suppose that there are two processes, with a request 1 happening before 2. *check the full proof on the slides*

🔗 Exercise

Show the fairness property is violated if the channel is not FIFO.

There is a point where we say "by FIFO it's not like this", and from there we can show an execution that would probably break the algorithm.

As a measuring complexity of the algorithms we use two measures:

- message complexity, the number of messages generated to reach a goal, in this case it's $3n$.

- time required for the algorithm to terminate. In sync systems, we simply count the time. In async systems we count each message as a unit of time, as well as the length of the longest chain of messages. *When a process sends a request, it takes one unit of time to receive the message (each process receives it as broadcast). This also works for the ACK, so we need two unit of times to enter the critical section.*

The worse case is given by $N+1$ time units, due to the chain of waiting given by releases by other processes $(N-1) + 2$ given by the request and the ack.